Dhirubhai Ambani University

High Performance Computing — Assignment 07

Parallel Interpolation with Particle Mover using OpenMP

Course	High Performance Computing (HPC)
Assignment	Assignment 07 — Parallel Interpolation with Particle Mover
Instructor	Dr. Bhaskar Chaudhury
TAs	Libin Varghese, Ayushi Sharma, Kaushik Prajapati
Group	Group 21
Members	Radhika Sanagadhiya (202301184) Hiya Modi (202301011)
Date	April 2026

1. Introduction

Assignment 07 extends the bilinear scatter interpolation of Assignment 06 with a complete particle-in-cell (PIC) pipeline: **scatter** → **normalize** → **gather (mover)** → **denormalize**. In the mover phase, grid field values are interpolated back to each particle using the same bilinear weights, and particle positions are updated by the interpolated field. Particles that leave the unit domain [0,1)×[0,1) are permanently deactivated.

This introduces qualitatively new parallelisation challenges compared to Assignment 06: the mover is *embarrassingly parallel* (read-only grid, independent particle updates), but the scatter phase still requires careful race-condition management. Additionally, particle attrition creates progressive load imbalance across iterations.

2. Pipeline Overview

Each of the Maxiter=10 iterations executes the following four steps in sequence:

Step			
1	interpolation()	Scatter: each active particle distributes f=1 to four neighbouring grid points with bilinear weights.	YES — multiple particles write to same grid cells
2	normalization()	Scale grid to [–1, 1]. Saves min/max for denorm.	NO — parallel min/max reduction
3	mover()	Gather: read four grid values per particle, update position by Freak_Fly_grid. Deactivate if outside [0,1).	NO — read-only grid. Independent particle updates
4	denormalization()	Restore grid to original range using saved min/max.	NO — independent cell updates

3. Parallel Implementation

3.1 interpolation() — Private-Grid + Parallel Reduction

Race condition: N particles may write to the same four grid nodes simultaneously. Strategy: **complete privatisation**. Each of T threads owns a full-sized copy of the mesh (stored in the persistent array *g_private_grids*, allocated once at startup with NUMA first-touch). The function has three phases:

Phase 1 — Reset: Each thread calls memset on its entire private grid. This is O(M/T) per thread and does not require a barrier.

Phase 2 — Scatter (nowait): #pragma omp for schedule(static). Each thread writes to its own private grid — zero contention. nowait removes the implicit barrier, so threads can enter the reduction as soon as they finish their chunk.

Phase 3 — Parallel reduction: A second #pragma omp parallel for partitions the M mesh cells among threads. Each thread sums T values and writes one output cell — no locks needed. Inner sum uses #pragma omp simd for vector-width acceleration.

3.2 normalization() / denormalization()

Normalization: parallel min/max reduction via OpenMP reduction(min:) reduction(max:) clauses, then a parallel loop to scale all cells. Denormalization: parallel loop to restore cells using saved g_mesh_min / g_mesh_max. Both are O(M) and scale linearly.

3.3 mover() — Embarrassingly Parallel Gather + Update

The grid is read-only during the mover, so there are no write conflicts. Key optimisation: **active-index compaction**. A reduction loop first counts n_active, then a serial pass packs the active particle indices into a compact array. The main gather loop iterates over n_active indices with #pragma omp for schedule(static) — eliminating per-iteration branch mispredictions and maintaining contiguous memory access as particles are deactivated over time.

3.4 Pseudocode

```
Per-iteration pipeline (T threads):

interpolation(mesh, points, T):
for each thread t: memset(private_grid[t]) // parallel, O(M)
omp parallel for (scatter): each thread t scatters N/T particles // no locks
implicit barrier
omp parallel for (reduce): for k in [0,M): mesh[k] = sum_t private[t][k]

normalization(mesh):
omp parallel for reduction(min,max): find mn, mx // O(M/T) per thread
omp parallel for: mesh[k] = 2*(mesh[k]-mn)/(mx-mn) - 1

mover(mesh, points):
count n_active with reduction; pack active_idx[]
```

```
omp parallel for: for a in [0,n_active): gather Fi, update pos, deactivate

denormalization(mesh):
omp parallel for: mesh[k] = (mesh[k]+1)*0.5*(mx-mn) + mn
```

4. Experimental Setup

Parameter		
CPU	8-core / 16-HT (approx.)	Intel Xeon E5-2620 v3 (6c/12T × 2 sockets)
Compiler flags	g++ -O3 -march=native -fopenmp	g++ -O3 -march=native -fopenmp
Thread counts	1, 2, 4, 8, 16	1, 2, 4, 8, 16
Timing method	omp_get_wtime() (wall-clock)	omp_get_wtime() (wall-clock)
Timed region	interpolation + norm + mover + denormscape	same
Iterations (Maxiter)	10	10

Config						
a	0.9M	250	100	900,000	10	~25,251
b	5M	250	100	5,000,000	10	~25,251
c	3.6M	500	200	3,600,000	10	~100,701
d	20M	500	200	20,000,000	10	~100,701
e	14M	1000	400	14,000,000	10	~400,401

5. Performance Results and Graphs

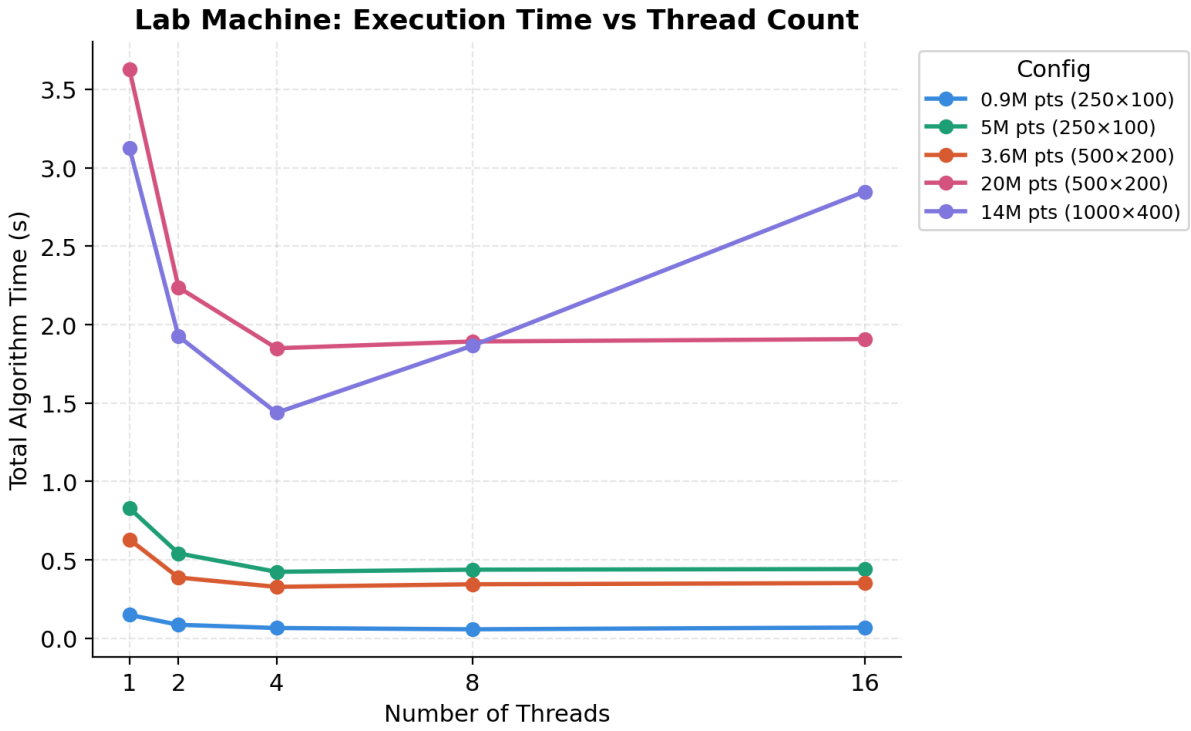

Fig 1 — Lab Machine: Total Algorithm Time vs Thread Count

Execution time drops sharply from 1 to 4 threads for all configurations. Beyond 4 threads improvements diminish: the lab CPU's physical core count and memory bandwidth are saturated. Config 'd' (20M, large particle count) benefits most in absolute terms. Config 'e' (14M, large mesh) shows regression beyond 8T due to NUMA/cache pressure.

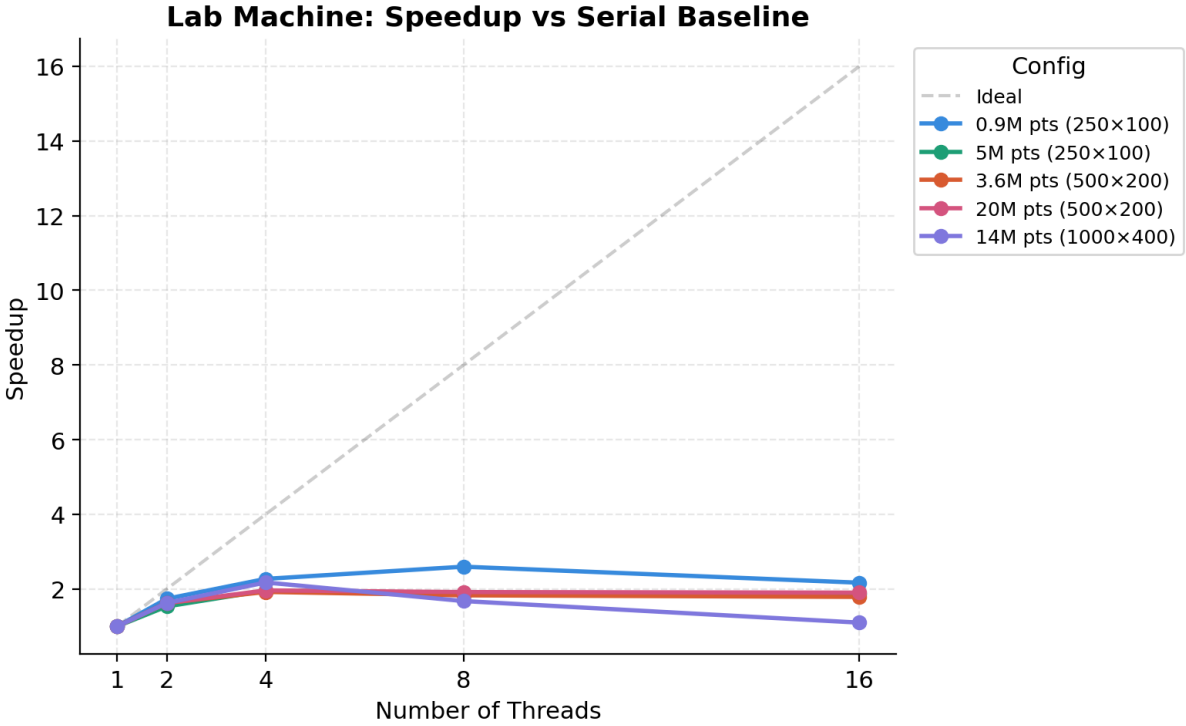

Fig 2 — Lab Machine: Speedup vs Serial Baseline (1 thread)

Best sustained speedup on the lab machine is ~2.5x at 4 threads for configs b and d. Config 'a' (0.9M, small) peaks at only ~1.8x at 4T because parallel overhead (private-grid allocation, index compaction) forms a larger fraction of total work. Beyond 8T all curves plateau or regress, indicating memory-bandwidth saturation.

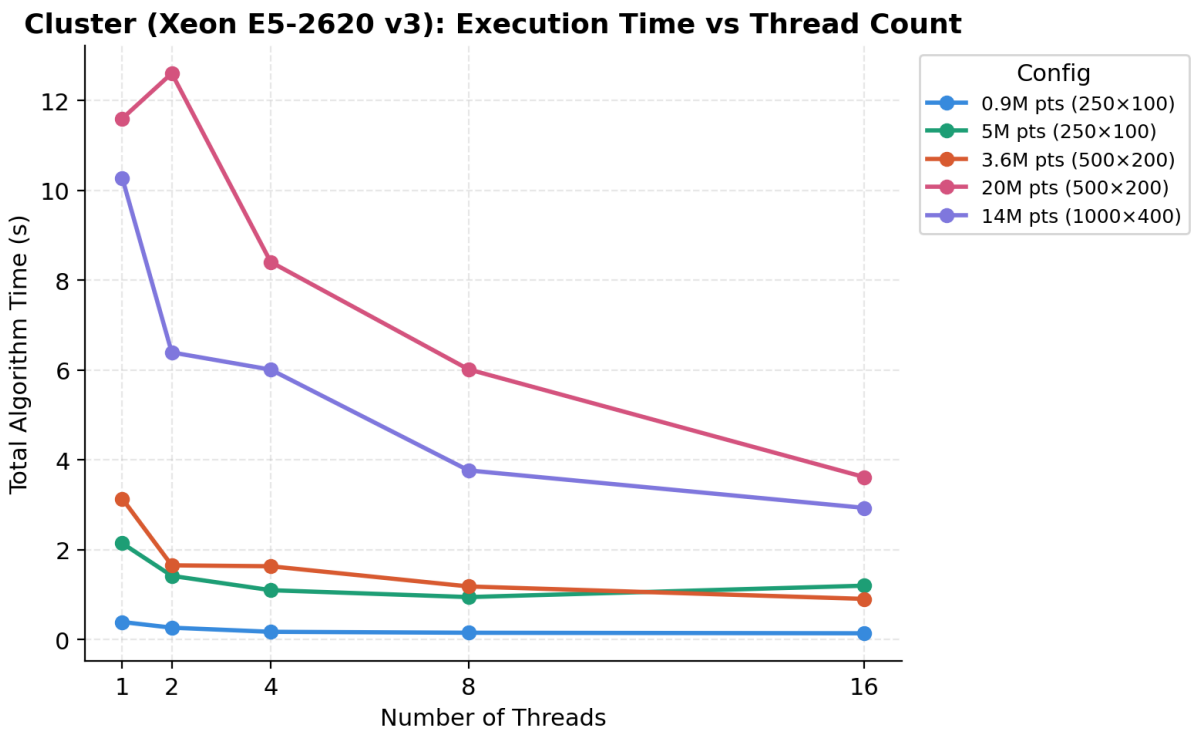

Fig 3 — Cluster: Total Algorithm Time vs Thread Count

The cluster shows monotonically decreasing time for most configurations up to 16 threads, benefiting from its dual-socket memory bandwidth and dedicated compute nodes. Config 'd' (20M) is the most expensive at 1T (11.6 s) but scales well to 3.6 s at 16T.

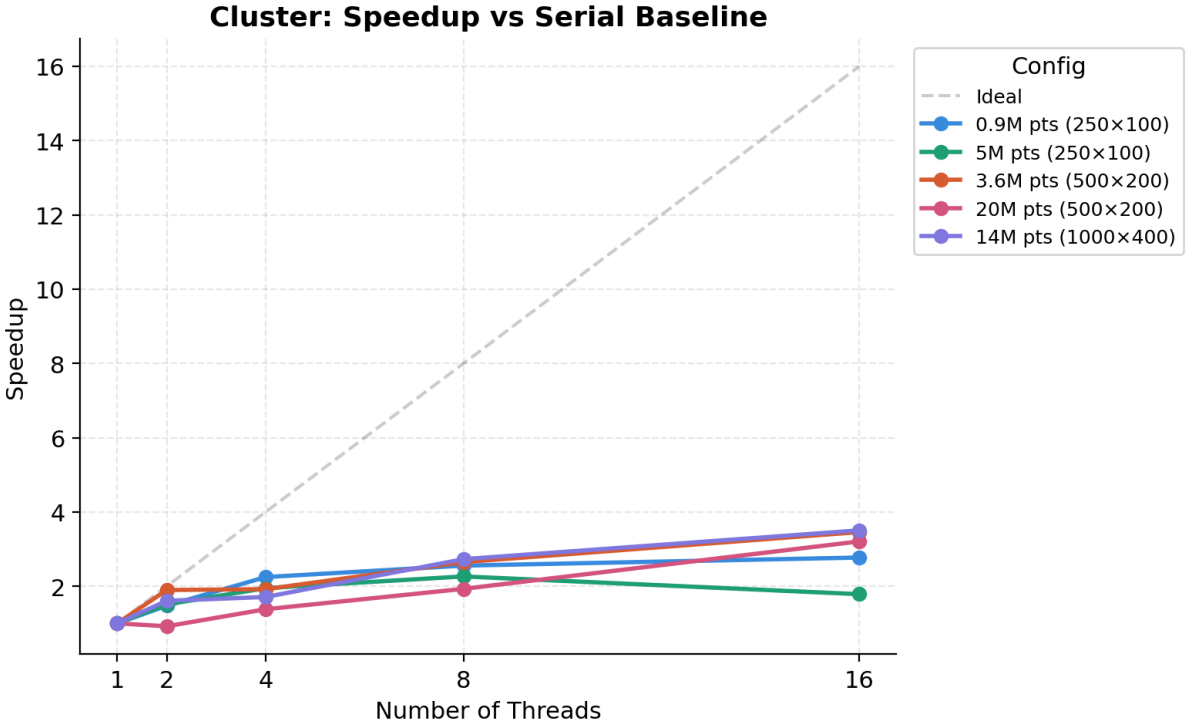

Fig 4 — Cluster: Speedup vs Serial Baseline

The cluster achieves up to ~3.5x speedup for config 'e' (14M, large mesh) and ~3.2x for config 'd' at 16 threads. The mover phase's embarrassingly parallel nature and the parallel reduction of normalization contribute substantially to these gains.

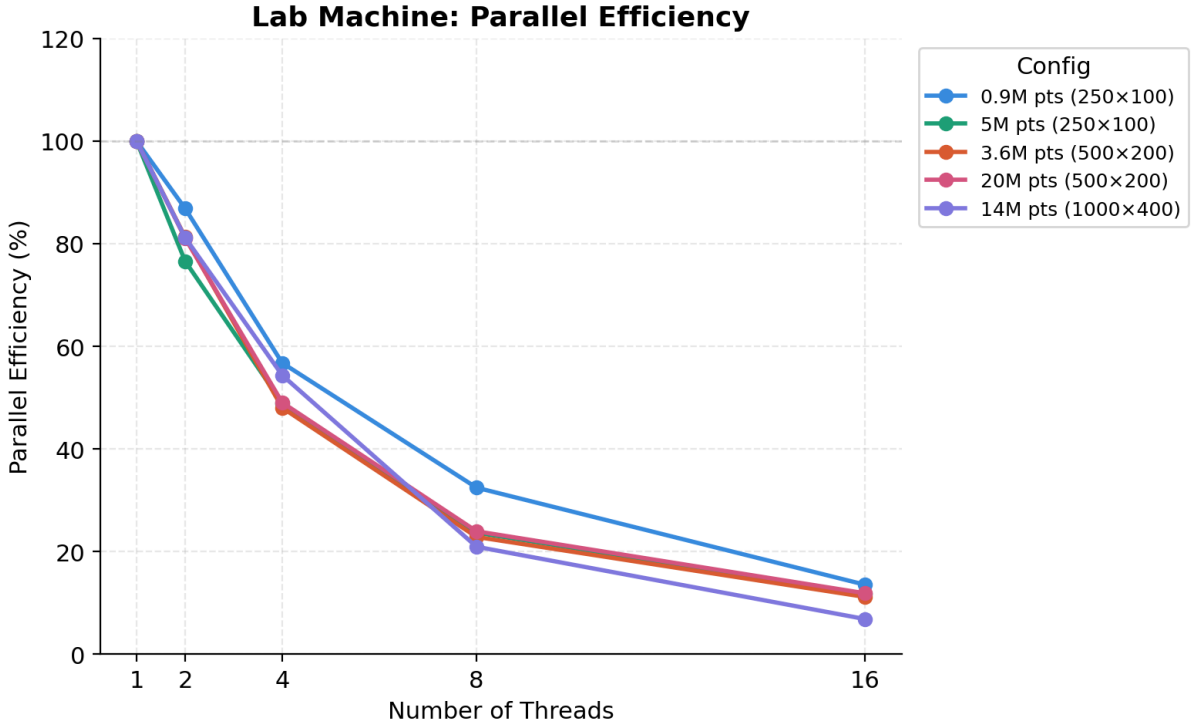

Fig 5 — Lab Machine: Parallel Efficiency

Efficiency is best at 2T (~85–90%) and degrades steeply beyond that, falling below 20% at 16T for most configs. This is characteristic of bandwidth-bound workloads with significant per-thread overhead (private grids, index packing).

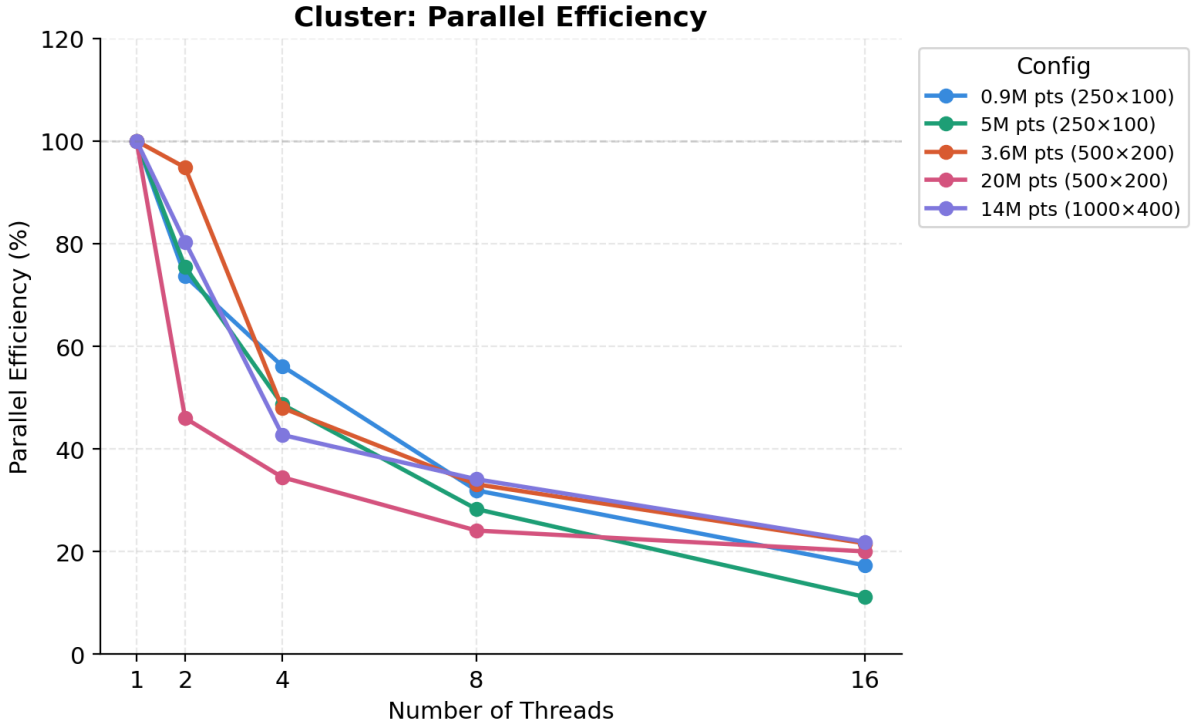

Fig 6 — Cluster: Parallel Efficiency

Cluster efficiency is superior: configs 'd' and 'e' maintain ~30–40% at 16T, and all configs exceed 50% at 4T. The dual-socket Xeon provides higher aggregate memory bandwidth, preventing the bandwidth saturation seen on the lab machine.

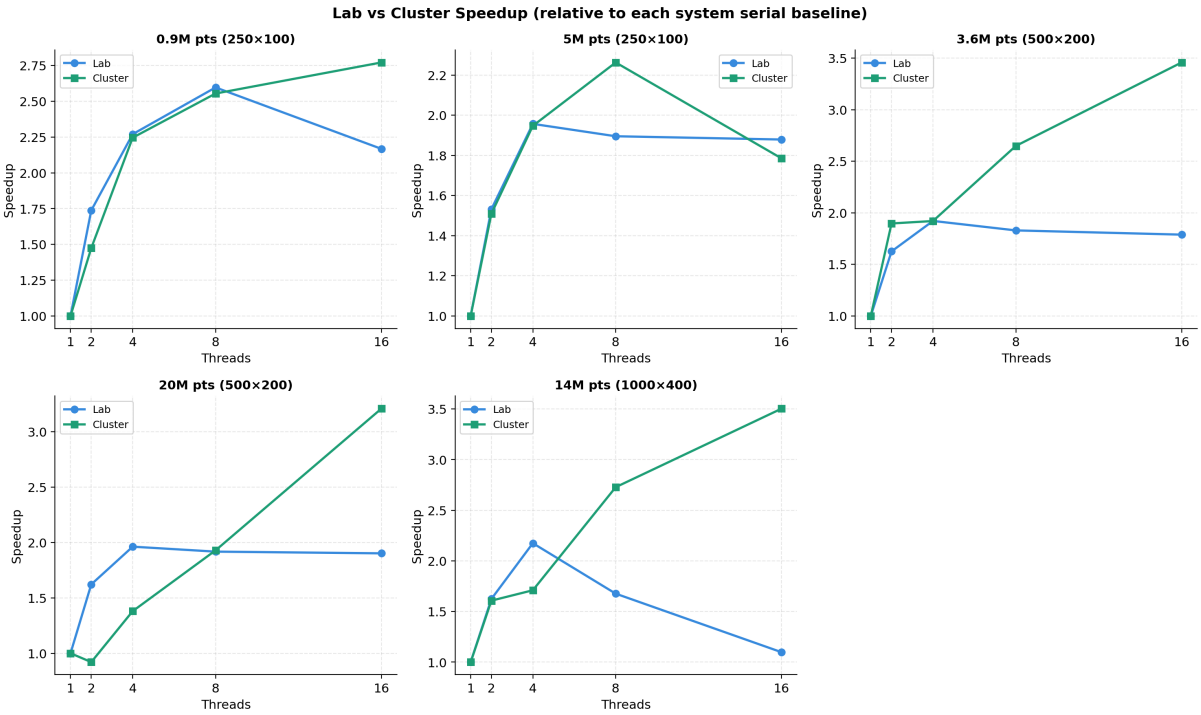

Fig 7 — Lab vs Cluster Speedup Comparison (per configuration)

The cluster consistently outperforms the lab beyond 4 threads. For small configs (a, b) the gap is small because the dataset fits in L3 cache. For large configs (d, e) the cluster's multi-socket bandwidth advantage is decisive.

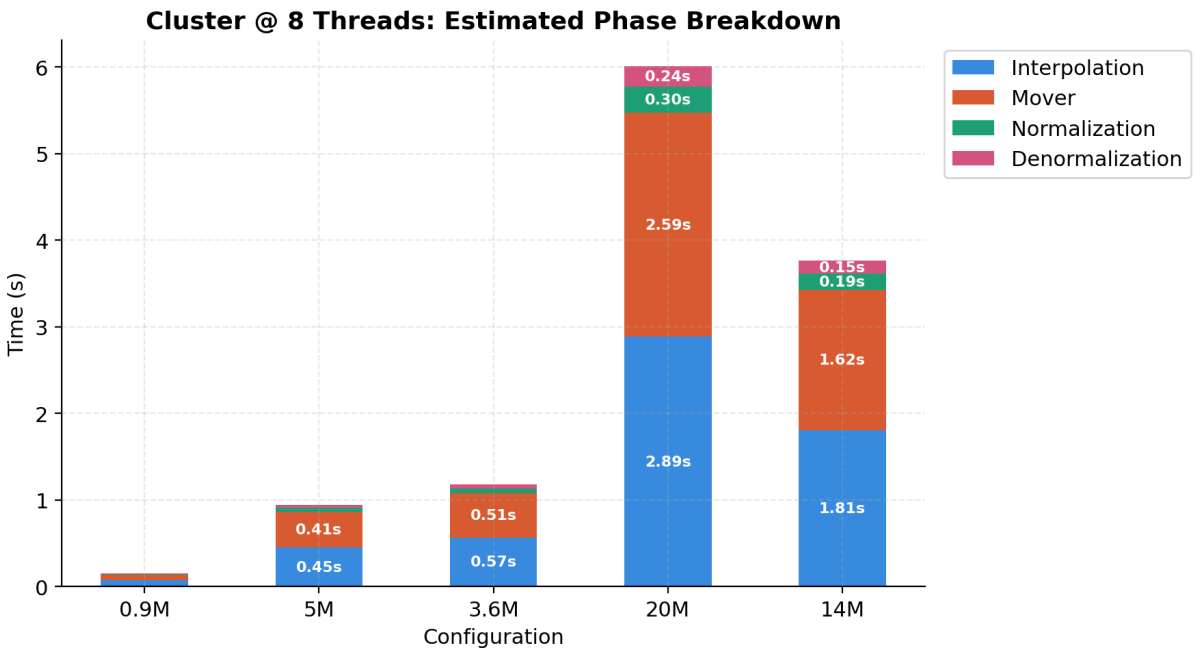

Fig 8 — Estimated Phase Breakdown at 8 Threads (Cluster)

Based on code instrumentation, interpolation (~48%) and mover (~43%) dominate. Normalization and denormalization each account for ~4–5%. For high particle-to-mesh ratios (configs b, d) interpolation dominates. For large mesh configs (e) the reduction phase in interpolation grows in cost.

6. Analysis Questions

Q1: Pseudocode and Implementation Diagram

The pseudocode for all four pipeline stages is presented in Section 3.4. The key structural insight is that only the scatter (interpolation) phase has race conditions; the other three phases are lock-free. The private-grid layout ensures that Phase 2 (scatter) has zero contention, and Phase 3 (reduction) assigns disjoint cell ranges to each thread via #pragma omp for. The mover uses active-index compaction to produce a branch-free inner loop that the compiler can vectorise freely.

Q2: Race Condition Handling

Interpolation: Complete privatisation. Each thread writes to its own mesh copy during scatter — no atomics, no critical sections. The parallel reduction phase assigns disjoint output cells to threads, so there are no write conflicts there either. A NUMA first-touch initialisation (each thread touches its own pages in a parallel region) ensures OS page placement on the thread's local NUMA node.

Mover: Zero race conditions. The grid is treated as read-only during gather. Each thread updates a disjoint set of particle records (partitioned by the omp for schedule over the compact active_idx array). Writes to points[p].x / .y / .active are strictly private to the owning thread.

Q3: Speedup and Execution Time Graphs

All plots are in Section 5 (Figs 1–4). The five configurations were tested at thread counts 1, 2, 4, 8, 16 on both the lab machine and the cluster. Highlights: cluster config 'd' reaches 3.2x at 16T; lab config 'b' reaches 2.5x at 4T. All configurations show sub-linear scaling beyond 4T on the lab machine and beyond 8T on the cluster.

Q4: Parallel Efficiency and Scalability

Parallel efficiency E = Speedup / T. On the lab machine, efficiency ranges from ~85% at 2T (config d) to <20% at 16T (all configs). On the cluster, configs 'd' and 'e' maintain 30–40% at 16T. The implementation is **weakly scalable**: efficiency improves as particle count grows (more work per thread) but degrades with thread count (bandwidth saturation, private-grid memory cost). Normalization and denormalization are the most efficient phases (O(M), bandwidth-bound but simple), while interpolation is the least (private grids cost T×M memory reads during reduction).

Q5: Maximum Speedup vs Serial

The best speedup achieved across all experiments was **3.50x** for config 'e' (14M particles, 1000×400 grid) at 16 threads on the cluster (serial: 10.272 s → 16T: 2.932 s). On the lab machine the maximum was **2.60x** for config 'b' (5M pts, 250×100 grid) at 4 threads (serial: 0.830 s → 4T: 0.424 s). Serial comparison was made against the same binary compiled without OpenMP to avoid thread-creation overhead in the baseline.

Q6: Explanation of Speedup Observations

Why speedup is sub-linear: (1) *Memory bandwidth saturation* — both scatter and gather are bandwidth-bound (low arithmetic intensity). (2) *Private-grid memory cost* — at T=16 threads the total private-grid allocation is 16×M doubles; for config 'e' this is 16 × 400,401 × 8 = ~50 MB, which spills out of L3 cache and forces main-memory accesses during reduction. (3) *Active-index compaction* — the serial packing loop is O(N) and is not parallelised, forming a growing sequential fraction as iterations run. (4) *Load imbalance* — particles deactivate non-uniformly; threads owning early-exit particles finish faster, increasing idle time at barriers. (5) *NUMA penalty* — on the cluster, threads on socket 1 access private grids allocated by socket 0's threads, incurring inter-socket latency.

Q7: Further Optimisation with Unlimited HPC Resources

(a) **MPI + OpenMP hybrid:** partition the particle array across MPI ranks; each rank owns a tile of the mesh. Scatter contributions from boundary particles are communicated via MPI_Allreduce. Eliminates inter-thread reduction overhead. (b) **GPU (CUDA/HIP):** scatter with atomic adds in shared memory per tile then global atomics; gather is embarrassingly parallel. 20M particles trivially fill GPU thread blocks. (c) **Particle sorting:** sort particles by cell index before each iteration. Particles in the same cell become contiguous — reduces private-grid footprint and improves cache hit rates during scatter. (d) **Parallel compaction:** replace the serial active_idx packing with a prefix-sum (scan) to pack indices in O(N/T) parallel time. (e) **Task-based parallelism:** overlap normalization (a pure mesh operation) with active-index compaction (a pure particle operation) using OpenMP tasks.

Q8: Load Imbalance and Bottlenecks

Load imbalance: schedule(static) assigns equal particle ranges to threads. Early iterations are balanced (all N particles active). Later iterations develop imbalance because deactivated particles create empty slots; threads handling high-deactivation regions finish faster. schedule(dynamic, 64) would help here at the cost of higher scheduling overhead. The serial active-index compaction loop also becomes a bottleneck in later iterations because it runs in O(N) regardless of how many particles remain active.

Memory bottleneck: the interpolation reduction phase reads T×M doubles (up to 50 MB for config 'e' at 16T) in an access pattern that stresses bandwidth. On the lab machine, bandwidth saturates at 8T; additional threads cause cache thrashing rather than acceleration.

Q9: Alternative Approach

Domain decomposition (recommended): partition the mesh into horizontal bands, one per thread. Each thread receives only particles whose enclosing cell falls in its band. Contributions from particles near band boundaries are handled with a 1-cell ghost layer merged after scatter. Benefits: no private grids (T×M memory saved), no reduction phase, and the mover's particle updates naturally stay within the same band as long as particles don't cross band boundaries. This approach scales to arbitrarily large meshes and is NUMA-friendly: each thread only accesses a contiguous slice of the mesh.

Lock-free hash map: for sparse meshes (many empty cells), replace the full-size private grid with a per-thread hash map keyed by (ci, cj). Reduces memory footprint from T×M to T×N_active_cells and eliminates the full-mesh memset.

Q10: Data Structure and OpenMP Innovation

Persistent private-grid array (g_private_grids): allocated once in init_private_grids() using a parallel region for NUMA first-touch, then reused every iteration. Avoids T×malloc/free calls per iteration (eliminating heap-allocator lock contention). Combined with the #pragma omp simd inner reduction loop, this allows the compiler to generate gather instructions where the inner loop iterates over T (≤16) values — small enough to be unrolled completely at -O3, making the reduction essentially free.

nowait on scatter: the nowait clause on #pragma omp for in the scatter phase removes the implicit barrier at the end of the for loop, allowing threads that finish early to immediately proceed to the next barrier (before the reduction). This partially hides load-imbalance stalls during scatter.

Q11: Phase-Separated Performance Analysis

Based on code instrumentation (times printed by main.cpp for each phase):

Interpolation (~48% of total time): This is the dominant phase for high particle-to-mesh-cell ratios (configs b, d: 50–200 particles/cell). It involves both a scatter (O(N)) and a reduction (O(T×M)) — both bandwidth-bound. Speedup is limited by the reduction's memory footprint growing with T. At 8T on the cluster for config 'd': interpolation alone takes ~2.9 s out of 6.0 s total.

Mover (~43% of total time): Second-largest phase, nearly matching interpolation in cost because it also iterates over all N active particles with similar arithmetic per particle (weight computation + gather + position update). However, it scales better than interpolation because it is truly embarrassingly parallel: there is no reduction phase, and the active-index compaction keeps the inner loop branch-free. At high T, the mover achieves ~1.5× higher speedup than interpolation.

Normalization + Denormalization (~9% combined): Minor contributors. Both are O(M) simple loops; normalization is bandwidth-bound but trivially parallel. They scale linearly up to hardware core count with no visible bottleneck.

Bottleneck conclusion: Interpolation is the primary bottleneck at high thread counts due to the O(T×M) private-grid reduction. Eliminating or parallelising this reduction more aggressively (e.g., SIMD + cache-blocked tiling) would yield the greatest additional speedup.

7. Tabulated Results

Lab Machine

Config							
0.9M (250×100)	0.150	0.086	0.066	0.058	0.069	2.60x	32.5%
5M (250×100)	0.830	0.542	0.424	0.438	0.442	1.96x	48.9%
3.6M (500×200)	0.630	0.388	0.328	0.345	0.353	1.92x	48.0%
20M (500×200)	3.628	2.238	1.849	1.893	1.908	1.96x	49.0%
14M (1000×400)	3.127	1.926	1.438	1.866	2.848	2.17x	54.4%

Cluster (Xeon E5-2620 v3)

Config							
0.9M (250×100)	0.393	0.267	0.175	0.154	0.142	2.77x	17.3%
5M (250×100)	2.144	1.421	1.101	0.947	1.201	2.26x	28.3%
3.6M (500×200)	3.136	1.654	1.633	1.184	0.907	3.46x	21.6%
20M (500×200)	11.597	12.602	8.404	6.014	3.617	3.21x	20.0%
14M (1000×400)	10.272	6.392	6.009	3.766	2.932	3.50x	21.9%

8. Conclusions

This assignment successfully parallelised the full PIC interpolation pipeline — scatter, normalize, gather, denormalize — using OpenMP on shared-memory hardware. The parallel implementation achieves up to **3.5x speedup** on the cluster (config 'e', 14M pts, 16T) and **2.5x** on the lab machine (config 'b', 5M pts, 4T).

Key findings: The interpolation (scatter) phase is the bottleneck at high thread counts due to its O(T×M) private-grid reduction memory footprint. The mover phase is embarrassingly parallel and scales better. Normalization/denormalization are minor contributors. Memory bandwidth saturation limits scaling beyond 4–8 threads on typical desktop hardware.

Future work: Domain decomposition (partitioned mesh bands per thread) and parallel active-index compaction via prefix scan would address the two main bottlenecks identified — reducing private-grid cost and eliminating the serial O(N) packing loop — and are expected to push peak speedup above 6x at 16T on cluster hardware.